Project Handbook — "What Makes a Global Top 50 Hit Last?"

> The complete story of this project, from the first commit to today: what it is, why every decision was made, how each part works, what went wrong and how it was fixed, its strengths and limits, and what to do next. Written so that anyone — a recruiter, a collaborator, or an AI assistant starting with no memory — can pick the project up and continue it safely.

Author and sole contributor: Neha Karamchandani (GitHub `n3h4a`)
Repository: https://github.com/n3h4a/spotify-powerbi-dashboard
Live dashboard: https://n3h4a.github.io/spotify-powerbi-dashboard/

Contents

1. What this project is
2. Where it started and why it changed
3. The data
4. Architecture at a glance
5. Data engineering: the warehouse
6. Data quality
7. Analytics: the nine business questions
8. Data science: survival analysis and prediction
9. Power BI, generated as code
10. The web dashboard
11. Testing
12. CI/CD and deployment
13. Hard problems and how they were solved
14. Bugs found along the way (and their fixes)
15. Key numbers
16. Strengths
17. Limitations
18. Future enhancements
19. How to run and change things
20. Rules for working on this repo
21. File map
22. Résumé and interview talking points

---

1. What this project is

An end-to-end data project on 18 months of daily **Spotify Global Top 50** snapshots (556 days, 27,800 chart entries, 797 songs, 345 artists). It answers one central business question — **what makes a hit last?** — from three angles, one per data role:

Role	What the project shows
Data engineering	A tested, reproducible DuckDB warehouse: bronze → silver → gold star schema → marts → ML features; SCD Type 2; 45 automated data-quality checks; CI that fails on stale outputs.
Data analytics	Nine business questions answered in SQL with statistical tests, 12 publication-quality figures, a Power BI report and a live web dashboard.
Data science	Kaplan-Meier survival curves, a Cox proportional-hazards model with assumption tests, and a leakage-safe classifier that predicts on day 7 whether a song will last 30 days (AUC 0.91).

The problem it solves: a label, playlist editor or marketing team has to decide early which new releases deserve promotion budget. The project shows that most chart runs are short (median 6 days), that the first week is far more predictive than launch-day signals, and turns that into a watchlist ranked by the probability of a song lasting.

2. Where it started and why it changed

Original state: a single Power BI file (`dashboard/spotify_dashboard.pbix`), the raw CSV and screenshots. It was a pretty, colourful dashboard, but nothing behind it was reproducible, tested or explainable — not enough to stand out for FAANG-level data roles.

Goal set by the owner: make the project résumé-worthy for data analytics, data engineering and data science roles, keep it beautiful and informative, and keep the owner as the only contributor.

How it evolved (in order):

1. **Phase 1 — Data engineering.** Built the Python package, SQL medallion layers, star schema, data-quality framework, tests and CI.
2. **Phase 2 — Analytics.** Nine business questions, statistical tests, figures, an insights report, marts and a DAX measure library.
3. **Phase 3 — Data science.** Survival analysis, Cox model, prediction with honest validation.
4. **Power BI as code.** Power BI Desktop can't run in a Linux sandbox, so the report is *generated* as a PBIP project (TMDL model + PBIR report) and validated against Microsoft's JSON schemas.
5. **Design port.** The owner found the first generated report "bland and plain", so the design of her original `.pbix` (theme, layout, colours, images) was parsed and ported onto the new model.
6. **Making it correct.** The owner reported things that didn't work (cards not updating, images missing, Taylor Swift showing everywhere, "2K" as a year, empty cards). Each was traced to a root cause in the data model or the visuals and fixed with a test so it can't come back. This also uncovered real data-identity problems (split recordings, "Various Artists" taking credit).

7. **Web dashboard + end-to-end tests.** Because Power BI can't be run here, a second front end was built — a static web dashboard — so every number could be *actually clicked through and tested* in a real browser against independent SQL.
8. **Deployment.** GitHub Pages via GitHub Actions, with screenshots captured from the live site on every deploy and embedded in the README.
9. **Polish.** Round artist images from each artist's top song cover; README screenshots from the live site; this handbook.

3. The data

`dataset/spotify-top-50-world.csv` — one row per (date, position):

Column	Notes
`date`, `position`	556 snapshots × 50 positions. 4 calendar days are missing.
`song`, `artist`	Artist is a credit string such as `"A & B"`.
`popularity`	Spotify's 0–100 score at that moment (not stream counts).
`duration_ms`, `album_type`, `total_tracks`, `release_date`, `is_explicit`	Attributes that can change over time (re-releases, deluxe albums, re-credits).
`album_cover_url`	Cover art on `i.scdn.co`.

Quirks that shaped the design:

- **Mixed line endings** (the first 51 lines LF, the rest CRLF) broke DuckDB's CSV sniffer.
- **Release dates at three precisions** (`2024`, `2024-05`, `2024-05-17`).
- **Artist credits joined with** `" & "`, but some real names contain `&` or commas (Earth, Wind & Fire; Tyler, The Creator).
- **The same recording under different lead credits** (Kanye West ⁻ ¥$ on 16 Feb 2024; "Various Artists" compilations).
- **Songs already charting on day 1** (left-censored) and **still charting on the last day** (right-censored), so naive "chart life" numbers are biased.
- **Missing days**, so "days" and "snapshots" are not the same thing.

4. Architecture at a glance

```
CSV ──► bronze (raw text + lineage)
          └► silver (typed, credits split, recordings resolved)
               └► gold star schema: fact_chart_entry, dim_track, dim_track_version (SCD2),
                  dim_artist, dim_date, bridge_track_artist
                     ├► marts: track / artist / daily performance
                     │      ├► analyses (q1–q9) ──► insights.md + figures 01–05, 11, 12
                     │      ├► Power BI PBIP (TMDL + PBIR, Parquet data)
                     │      └► web dashboard JSON ──► GitHub Pages
                     └► ml.track_features ──► survival + prediction ──► model_report.md + figures 06–10
          45 data-quality checks run against every layer ──► data_quality_report.md
```

Why DuckDB: an in-process analytical database — real SQL (window functions, QUALIFY, Parquet I/O) with zero infrastructure, so anyone can clone and run the whole thing in a minute. The SQL is standard enough to move to Snowflake/BigQuery/dbt later.

Why medallion layers: each layer has one job (land → clean → model → serve), so a bug is traceable to one place, and raw data is never overwritten.

Why plain SQL files + a small Python runner instead of dbt: fewer moving parts to explain, and the runner shows the mechanics (templating, ordering, idempotent rebuilds) explicitly. The layout mirrors dbt, so migration is straightforward.

5. Data engineering: the warehouse

All SQL is in `sql/`, run in order by `src/spotify_pipeline/pipeline.py` (`STEPS`). Every model is `CREATE OR REPLACE` — rebuilding is idempotent and deterministic.

Bronze — `sql/bronze/raw_chart_snapshots.sql`

Reads the CSV with an explicit dialect (`delim`, `quote`, `escape`, `strict_mode=false`, `all_varchar=true`, `filename=true`). Everything stays text so nothing is lost; lineage columns record the source file. A quality check reconciles the row count against Python's `csv` module.

Silver — `stg_chart_entries.sql`, `int_chart_entries.sql`

- `TRY_CAST` typing; `release_date` plus `release_date_precision` (day / month / year).
- **Artist credit splitting** (`artists.py: split_artist_credit`): protected names (`seeds/protected_artist_names.csv`) are masked longest-first, the credit is split on `" & "`, duplicates are removed case-insensitively.
- **Track identity = (title, lead artist).** Recordings whose lead credit changed are merged when the title and `duration_ms` match exactly (`canonical` CTE: prefer a real artist over "Various Artists", then the most-charted lead). This turned 807 apparent tracks into 797 real ones.
- `version_attributes_hash` detects when a track's attributes change.

Gold — star schema

Table	Grain / purpose	Notes
`fact_chart_entry`	one row per date × position	`previous_position`, `position_change`, `entry_status` (debut / re-entry / returning), `chart_days_to_date`, `current_streak_days`, `peak_position_to_date`. Movement is measured **between snapshots**, not calendar days.
`dim_track`	one row per song	`track_label` (title, plus artist only when titles collide, e.g. three different "Daylight"s), `chart_rank` (most chart days first; used as a default sort so nothing is alphabetical by accident).

Table	Grain / purpose	Notes
`dim_track_version`	SCD Type 2	Gaps-and-islands over the attribute hash; `valid_from` / `valid_to` (9999-12-31 for current). 1,089 versions.
`dim_artist`	one row per artist	Most frequent spelling wins.
`dim_date`	whole calendar years	`has_chart_snapshot`, `is_holiday_season`.
`bridge_track_artist`	many-to-many	`attribution_weight = 1/n`, lead artist first. "Various Artists" never shares credit with a real artist.

Keys are deterministic MD5 hashes (`md5('track|' || lower(title) || '|' || lower(lead))`), so rebuilds give identical keys and diffs are meaningful.

Marts

- `mart_track_performance` — one row per song: debut, peak, days to peak, peaked-on-debut, total chart days, re-entries, longest streak, days at #1 / in the top 10, popularity, debut attributes, release age at debut (new ≤ 30 days / recent ≤ 365 / catalog), censoring flags (`is_left_censored`, `is_right_censored`, `has_full_180_day_window`), `chart_days_first_180`, chart-life bucket, current cover, and `is_holiday_track` (≥ 90 % of chart days in the holiday season **and** released over a year before debut **and** not in `seeds/not_holiday_tracks.csv`).
- `mart_artist_performance` — weighted chart days, share of chart, rank, #1 days, best position.
- `mart_daily_chart` — debuts, re-entries, turnover, artist HHI concentration, holiday share, average popularity.

ML features — `sql/ml/track_features.sql`

Point-in-time features only: launch-day facts, week-one trajectory (`w1_position_slope`, day-7 position = 51 if off the chart), and artist history **strictly before** the debut. Label `lasts_30_days` is null when a song lacks a full observation window. Survival columns: `chart_life_days`, `chart_life_ended` (absent for the final 14+ days).

6. Data quality

`src/spotify_pipeline/quality.py` — **45 checks** (40 errors that fail the build, 5 warnings), each a SQL assertion that returns failing rows, with a sample shown in `docs/data_quality_report.md`. They cover:

- **Reconciliation:** raw rows = bronze rows = fact rows; marts reconcile to the fact table.
- **Keys:** uniqueness, foreign keys, one row per date × position.
- **SCD2 integrity:** no overlapping or gapped versions, exactly one current version.
- **Bridge:** weights sum to 1 per track; exactly one lead; Various Artists never shares credit.
- **Movement logic:** `position_change` consistent with `previous_position`.
- **Identity:** no split recordings remain; `track_label` unique.
- **ML features:** no feature uses data after the debut window.

The report is regenerated by `make check`; CI fails if the committed report is stale. Tests also corrupt data on purpose to prove each check fires.

7. Analytics: the nine business questions

SQL lives in `sql/analyses/` (`cohort.sql` + `q1`–`q9`); `insights.py` runs them, adds statistics and figures, and writes `docs/insights.md` and `site/data/insights.json`.

Method choices: a **fair-window cohort** (songs that debuted inside the data with a full 180 days to observe — 571 songs) removes censoring bias; **Mann-Whitney U** with rank-biserial effect size (chart days are heavily skewed, so no t-tests); **Spearman ρ** for monotonic relationships; **HHI** for concentration.

#	Question	Answer
1	How long do hits last?	Median 6 days; 53 % last a week or less.
2	How concentrated is the chart?	Top 10 % of songs hold 57 % of all slots.
3	Which song traits matter?	Climbers last 31 days vs 3 for peaked-on-debut; collaborations make no difference (6 vs 6, p = 0.96); slow burners 21 vs 6.
4	Album takeovers	30 of 50 slots on 2024-04-21 (HHI 3,669 vs median 309); 5 takeovers.
5	The holiday cliff	Christmas songs 94 % of the chart on 26 Dec 2023 → 4 % on 28 Dec; 51 holiday songs.
6	Who dominates?	Taylor Swift 6.7 % of the chart (1,871 weighted days, 85 songs, 2.1× the next artist); top 10 artists hold 27 %.
7	Release timing	Median 2 days from release to debut.
8	Release vs debut weekday	66 % released on Friday; 76 % debut on a weekend.
9	#1 reigns	Die With A Smile 77 days at #1 (57 in a row); Seven 75 days over 10 runs. 37 songs reached #1.

Recommendations derived: judge a release on its first week, not launch day; budget for climbers rather than big debuts; plan around Friday releases and album-week takeovers; treat holiday catalog as a seasonal, predictable inventory.

8. Data science: survival analysis and prediction

`src/spotify_pipeline/modeling.py` → `docs/model_report.md`, figures 06–10, `site/data/model.json`.

Why survival analysis: chart life is a time-to-event with censoring (songs still charting when the data ends). Treating those as "ended" would underestimate longevity; survival methods handle it correctly.

- **Kaplan-Meier** (statsmodels `SurvfuncRight`) curves by segment, compared with **log-rank**.
- **Landmark analysis at day 7:** only songs still charting on day 7 are compared, grouped by week-one behaviour — avoids immortal-time bias. Result: 57 vs 8 days remaining.
- **Cox PH** (`PHReg`, Efron ties): 5 of 9 covariates significant, Harrell's C = 0.65. **Proportional-hazards test:** Pearson correlation of Schoenfeld residuals with rank(time) (Grambsch–Therneau style). An earlier Spearman version had a 52 % false-positive rate in simulation; the final test was calibrated by simulation to 4 % false positives and 100 % power. Two violations remain (debut position, artist prior days) and are reported honestly.
- **Classifier:** "will this song last 30 days?" Logistic regression vs HistGradientBoosting, chosen by 5-fold CV inside the training set. **Time split:** train on debuts before 2024-01-01, test on 2024 debuts — never random, because the future must not leak into the past.
  - Week-one model AUC **0.91** [0.86–0.95]; launch-day model 0.69 [0.61–0.77]; baseline 0.61.
  - Paired bootstrap CIs (2,000 resamples); permutation importance.
  - Top-20 % precision 82 % vs a 29 % base rate (2.8× lift).
  - A **watchlist** of recent debuts ranked by predicted probability.
- **Leakage test:** a test changes future rows and asserts the features don't move.

9. Power BI, generated as code

Files: `powerbi/Spotify Charts Analytics.pbip`, `.SemanticModel/` (TMDL), `.Report/` (PBIR), `powerbi/data/*.parquet`, `powerbi/measures.dax`. Generated by `powerbi.py` and `powerbi_report.py`; guide in `docs/powerbi_guide.md` with the numbers each page should show.

Why as code: Power BI Desktop can't run on Linux, and binary `.pbix` files can't be reviewed in Git. PBIP is Microsoft's text-based project format — diffable, testable, reproducible. Every file is validated against the official Microsoft Fabric JSON schemas.

Semantic model: 9 tables, star relationships, 40 DAX measures. Key decisions:

- `mart_track_performance` ↔ `dim_track` is **one-to-one, bidirectional** and the bridge ↔ `dim_track` is bidirectional, so selecting a song or artist filters everything (this fixed "cards not updating"). A test checks the model has no ambiguous filter paths.
- **Focus-song measures** (`VAR FocusRank = MIN(dim_track[chart_rank]) …`) so Songs-page cards show one song, not a sum of all songs.
- `track_label` sorts by `chart_rank`, so the default song is the biggest hit, not whatever is first alphabetically (this fixed "Taylor everywhere").
- Release Year is text (`FORMAT(…, "0")`), so it can't be shown as "2K".

- Artist share-of-chart removes filters on the related tables from its denominator.
- Data loads through Power Query from the GitHub raw URL by default (a `DataLocation` parameter switches to local files).

Design port: her original `.pbix` (never modified — it is the design master) is unzipped and its UTF-16 `Report/Layout` parsed. Each visual's fields are remapped to the new model (`FIELD_MAP`, `BINDINGS` keyed by page, type and x/y position; `REPLACEMENTS` swap visuals that no longer fit). Formatting is kept property by property; colour rules that referenced removed measures become a solid colour; stale scope rules are pruned; cards use full numbers; a real TopN filter drives the Top 5 chart; the three album-cover "player" slicers don't filter each other.

10. The web dashboard

`site/` — static HTML/CSS/vanilla JS with ECharts 5.5.1 (vendored under `site/vendor/` with its Apache-2.0 licence). Live at https://n3h4a.github.io/spotify-powerbi-dashboard/.

Why a second front end: it can be opened by anyone (recruiters don't have Power BI), and it can be tested end-to-end in a real browser, which Power BI can't in this environment.

- Data: `site/data/dashboard.json` (~0.9 MB, built by `web.py`), `insights.json`, `model.json`. The browser computes every number from raw entries using the same definitions as the DAX.
- Pages (hash routes): **Overview** (KPIs, concentration over time with annotations, top artists, longest runs, #1 reigns, chart-life buckets, album vs single, debuts per month), **Artists** (list with round top-song covers, hero, KPIs, monthly share, songs, discography), **Songs** (hero, badges, chart journey on an inverted axis, popularity, SCD2 version history), **Insights** (the nine questions), **Predictions** (model KPIs, watchlist, figures).
- A period filter applies across pages.
- Covers that fail to load fall back to a coloured initial tile.
- `document.body.dataset.render` increments on each render so tests never read a stale page.

11. Testing

`pytest` — 103 tests in `tests/`:

- **Unit:** credit splitting, CLI, pipeline steps.
- **Data quality:** every check fires on deliberately corrupted data.
- **Edge-case fixture** (`conftest.py`): mixed newlines, protected names, re-credits.
- **Known facts** (`test_known_facts.py`): hand-verified truths, e.g. Espresso = 226 days, peak 1, 31 days at #1; Die With A Smile 77 days at #1; Taylor Swift 1,871 weighted days and 85 songs; CARNIVAL's lead is ¥$; Beautiful Things is credited only to Benson Boone; three distinct "Daylight"s; I Wanna Be Yours rank 1 with 548 days.
- **ML:** future-data leakage test; PH-test calibration by simulation.
- **Power BI:** every field and DAX reference resolves, TMDL syntax, M-variable shadowing, files not stale, navigation, images, theme, no ambiguous relationships, players, label fit, full numbers, colour rules, TopN.
- **Web end-to-end** (`test_web_e2e.py` → `site/tests/e2e.cjs`, Playwright): every period, all 345 artists and all 797 songs, plus clicks and interactions, compared against independent SQL at display precision.

12. CI/CD and deployment

`.github/workflows/ci.yml` on every push:

1. `test` — lint (ruff) + tests on Python 3.10 and 3.13.
2. `pipeline` — full build, checks, insights, model, web export; then `git diff --exit-code` on the generated reports and JSON, so committed outputs can never drift from the code.
3. `web` — Node 22 + Playwright end-to-end tests.

`.github/workflows/pages.yml` on pushes to `main` touching `site/**`, manually, and weekly (Mondays 06:17 UTC): serves the site, captures screenshots of every page (`site/tests/screenshots.cjs`) into `site/screenshots/`, and deploys to GitHub Pages. The README embeds those screenshots from the live URL, so they always match the deployed site — and no bot commits are needed (keeping `n3h4a` the only contributor).

Why deployment is simple here: the site is static files; GitHub serves them. A real production deployment would add a live data source, scheduled refreshes, an API, monitoring and access control.

13. Hard problems and how they were solved

Problem	Why it matters	Solution
Mixed CRLF/LF line endings	DuckDB's sniffer mis-parsed the file	Explicit dialect + row-count reconciliation check
Censoring (songs already/still charting)	Naive chart life is biased	Censoring flags, 180-day fair cohort, survival analysis
Missing snapshot days	"Days" ≠ "snapshots"	Movement measured between snapshots; streaks via gaps-and-islands
Changing attributes	A song's album/explicit flag/credit changes	SCD Type 2 `dim_track_version`
Collaborations	Credit must not be double-counted	Bridge table with 1/n weights; protected names
Split recordings (re-credits, compilations)	One song counted as two	Merge by title + exact duration; DQ check `no_split_recordings`
"Various Artists" stealing credit	Real artists lost 50 % share	Excluded unless it is the only credit; DQ check
Same title, different songs	Ambiguous labels	`track_label` adds the artist only when titles collide
Holiday songs	Distort longevity stats	Rule (season share, release age) + exception seed
Future leakage in ML	Inflated accuracy	Point-in-time features,

Problem	Why it matters	Solution
		time split, leakage test
Power BI can't run here	Can't click-test	PBIP as code + schema validation + a tested web twin

14. Bugs found along the way (and their fixes)

Kept on purpose — they show real debugging, and they explain why some code looks the way it does.

- DuckDB 2.0 "unfinished result" → read results with `fetchall()`.
- `regr_slope` returns NaN (not NULL) for one point → `CASE WHEN count(*) < 2 THEN 0.0`.
- Holiday rule flagged new November releases → added a release-age condition; then a seed for "Heather" (Conan Gray), a winter-themed but non-holiday song.
- PH test with Spearman had 52 % false positives → Pearson vs rank(time), verified by simulation.
- An M variable named `File` shadowed `File.Contents` → renamed `FileName`; test added.
- Porting dropped whole formatting blocks when one colour rule referenced a removed measure → cards showed dark text on dark background → fixed at property level; FillRule → solid colour.
- Positional metadata put "display units: none" on the wrong field → Release Year showed "2K" → text year + full-number cards.
- Long labels were truncated → shortened, with a label-fit test.
- Mart-based axes couldn't filter other visuals → 1:1 bidirectional relationship.
- Default selection was alphabetical (`"Slut!" (Taylor's Version)` everywhere) → `chart_rank` sort.
- Songs-page cards summed all songs → focus-song measures.
- Share of chart showed 100 % for a selected song → fixed denominator.
- A `GROUP BY ALL` + `QUALIFY` fix for Various Artists silently did nothing — the new DQ check caught it → rewritten with `NOT EXISTS`.
- Non-deterministic outputs failed CI's diff check (ties in q3; unordered credits query) → explicit tie-breakers and `ORDER BY`.
- An e2e race read the previous artist's page → render counter.
- JavaScript vs Python rounding of 85.25 differed → compare exact values within display precision.
- Lazy images were unloaded when tested → force eager loading.
- `make help` hid targets with digits → regex fixed.
- First Pages deploy failed because Pages wasn't enabled → enabled, re-ran, live.

15. Key numbers

Metric	Value
Snapshots / entries	556 / 27,800
Songs / artists / versions / bridge rows	797 / 345 / 1,089 / 986
Songs with an observed debut / fair cohort	747 / 571

Metric	Value
Songs that reached #1	37
Data-quality checks / tests / DAX measures	45 / 103 / 40
Model AUC (week one / launch day / baseline)	0.91 / 0.69 / 0.61
Power BI expected values	Overview 797 · 89.6 · 345 · 40.2 %; Sabrina Carpenter 14 · 90.9 · 51; Espresso 226 days, peak 1, 31 days at #1, 2024

16. Strengths

- **End to end:** raw CSV → warehouse → analysis → ML → two dashboards → live site.
- **Correctness is enforced, not claimed:** 45 DQ checks, 103 tests, known-fact tests, browser tests over every artist and song, CI that fails on drift.
- **Reproducible and deterministic:** one command rebuilds everything with identical output.
- **Honest statistics:** censoring handled, non-parametric tests, effect sizes, bootstrap CIs, time-based validation, assumption tests with their violations reported.
- **BI as code:** a Power BI project that is diffable, schema-validated and generated.
- **Owner's design preserved:** her original look carried over, not replaced.
- **Live, shareable demo** with screenshots that refresh themselves.

17. Limitations

- One chart (Global Top 50), 18 months; rank and popularity, **not stream counts**.
- Findings are **associational**, not causal (no marketing spend, playlisting or audio data).
- Small ML sample (hundreds of songs); results may not transfer to other markets or eras.
- Artist history only exists inside the dataset window.
- The Power BI report was validated statically; it has not been opened in Power BI Desktop in this environment, and README Power BI images are of the original design.
- The design port depends on her `.pbix` layout coordinates (`BINDINGS` keyed by x/y) — moving a visual in the `.pbix` means updating the bindings.
- Two Cox covariates violate proportional hazards.

18. Future enhancements

Roughly in order of value:

1. **Real Power BI screenshots** of the generated PBIP (needs the owner to open it in Desktop).
2. **Stratified or time-varying Cox model** for the two PH-violating covariates.
3. **More data:** more countries/charts, a longer window, and stream counts (e.g. Kworb) to model magnitude, not just rank.
4. **Audio features** by fuzzy-matching a public dataset (Spotify's audio-features endpoint is closed to new apps since Nov 2024; the basic Web API is free for metadata).
5. **Scheduled ingestion + incremental loads** (GitHub Actions cron or Airflow/Dagster), with orchestration and freshness checks.
6. **dbt migration** of the SQL layers (the layout already mirrors dbt).

7. **Cloud warehouse** (BigQuery/Snowflake) to show scale skills.
8. **An API** (FastAPI) if live queries are needed.
9. **An interview-prep sheet** with likely questions and answers about each design choice.

19. How to run and change things

```
python -m venv .venv && source .venv/bin/activate
make install        # pip install -e ".[dev,analysis,ml]"
make all            # build + check + export + insights + model + powerbi + web
make test           # pytest
make lint           # ruff
make serve          # http://localhost:8000 (web dashboard)
make e2e            # Playwright end-to-end tests (needs node + npm ci --prefix site)
```

CLI equivalent: `spotify-pipeline build|check|export|insights|model|powerbi|web|all`.

Typical change workflow:

1. Change SQL in `sql/` or Python in `src/spotify_pipeline/`.
2. `make all` to regenerate every output (reports, JSON, PBIP).
3. `make lint test` (and `make e2e` if the web dashboard or its data changed).
4. Commit **the regenerated files too** — CI fails if `docs/data_quality_report.md`, `docs/insights.md`, `site/data/dashboard.json` or `site/data/insights.json` are stale.
5. Add a known-fact test for any number a person would check by eye.

Where to change what: | To change… | Edit | |---|---| | A business rule (holiday, identity, credits) | `sql/silver`, `sql/marts`, `seeds/` | | A data-quality check | `src/spotify_pipeline/quality.py` | | A question or figure | `sql/analyses/` + `insights.py` | | The model | `sql/ml/track_features.sql` + `modeling.py` | | A DAX measure or relationship | `powerbi.py` (`MEASURES`, `RELATIONSHIPS`) | | Power BI visuals | `powerbi_report.py` (`BINDINGS`, `REPLACEMENTS`) — never edit the generated PBIP by hand | | The web dashboard | `site/app.js`, `site/style.css`, data shape in `web.py` | | Deployment / screenshots | `.github/workflows/pages.yml`, `site/tests/screenshots.cjs` |

20. Rules for working on this repo

1. **The owner, `n3h4a <nehakaramchandani2005@gmail.com>`, is the only author and committer.** Commits are unsigned. Never add `Co-Authored-By`, session links, "Generated with …" or any other attribution to commits, pull requests, code, docs or comments.
2. Never modify `dashboard/spotify_dashboard.pbix` — it is the design master.
3. Never hand-edit generated files (`docs/*_report.md`, `docs/insights.md`, `site/data/*.json`, the PBIP); change the generator and rerun.
4. Keep outputs deterministic (explicit `ORDER BY` and tie-breakers).
5. Every fix gets a test or a DQ check so it can't regress.
6. Charts follow the validated palette (light figures: blue `#2a78d6`, orange `#eb6834`, aqua `#1baf7a`; web: green `#1aac4e` marks on `#181a23`), no dual axes, direct labels.
7. Don't open pull requests unless the owner asks; work lands on `main` by fast-forward.

21. File map

```
dataset/                raw CSV
seeds/                  protected artist names, holiday exceptions
sql/bronze|silver|gold|marts|ml|analyses
src/spotify_pipeline/   config, artists, pipeline, quality, insights, modeling,
                        powerbi, powerbi_report, web, cli
tests/                  103 pytest tests (+ web e2e)
docs/                   this handbook, data model, data dictionary, Power BI guide,
                        generated reports, figures/01–12
```

```
powerbi/                generated PBIP + Parquet data + measures.dax
dashboard/              the owner's original .pbix (design master)
site/                   web dashboard, vendored ECharts, Playwright scripts
.github/workflows/      ci.yml, pages.yml
Makefile, pyproject.toml
```

22. Résumé and interview talking points

Résumé bullets (example):

- Built a tested DuckDB medallion warehouse (star schema, SCD Type 2, bridge table) over 27.8 K daily Spotify chart entries, enforced by 45 automated data-quality checks and CI.
- Answered 9 business questions with censoring-aware SQL and non-parametric statistics; found 64 % of songs peak on debut and climbers last 10× longer (31 vs 3 days).
- Built survival models (Kaplan-Meier, Cox PH with calibrated assumption tests) and a time-split, leakage-tested classifier predicting 30-day chart survival on day 7 (AUC 0.91, 2.8× lift).
- Generated a Power BI project as code (TMDL + PBIR, 40 DAX measures) and a live web dashboard, verified by Playwright tests across all 797 songs and 345 artists.

Questions to be ready for: why DuckDB and not Spark; what SCD2 is and why it's needed here; how the bridge weights avoid double counting; what censoring is and how it biases averages; why Mann-Whitney instead of a t-test; why a time split instead of a random split; what the PH assumption is and what to do when it fails; what AUC means and why precision-at-top-20 % matters to a business; how CI guarantees outputs aren't stale.